

The Participant — How Proxy Contributes on Its Own

The heart of the product: not a bot you ask, but a colleague who notices. Part A is what proactive contribution IS and every capability it unlocks. Part B is the agent design in plain terms — one mind with two halves and the loop it runs. Part C is the build, component by component, with the actual prompts. Part D is how it stays cheap and fast. This is the longest and most important of the deep dives; nothing is glossed.

Every meeting tool waits to be asked. Proxy also does the thing no other participant-shaped tool does: it *notices*. Unprompted, at exactly the right moment, it catches the wrong number, flags the contradiction, runs the check nobody thought to run — and only ever when it has something genuinely useful and provable to add. This document is how that judgment is built so it earns the room's attention instead of spending it.

THE ONE IDEA TO HOLD

The best-informed participant in any room is always quietly forming what they *would* say. The only real decision is whether to say it out loud. So Proxy is built the same way — it **always forms the thought first, then filters the finished thought**, never the other way around. One mind runs two halves: a **fast half** that drafts its best possible contribution on every change in the room, and a **deep half** that takes the worthy drafts and makes them real — proven with a test run, a live number, a precedent pulled. Between them sits a disciplined gate that decides what actually reaches the room, and a five-level dial the user controls. Everything below is how each of those parts works and how we make it cheap enough to run all meeting long.

PART A — THE PRODUCT

What proactive contribution is, and everything it makes possible

The capabilities a product person needs to picture — written before any implementation detail.

Proxy's proactive half is one plain promise: **it is the only participant in the room that can check what is being said against what is actually true in the company's own systems — before it speaks**. A human colleague, however sharp, cannot pull the live error rate mid-sentence, cannot run

the migration against synthetic traffic while the debate continues, cannot remember every number said in every meeting for the last year. Proxy can. But raw capability is not the product. A tool that *could* interrupt on everything and does is worse than useless — it gets muted in a day. The product is the **discipline**: superhuman ability held behind a gate strict enough that when Proxy does speak, the room turns to listen.

THE EQUATION THE WHOLE ENGINE COMPUTES

Superhuman participation = UNDERSTANDING × CAPACITY, held behind a disciplined GATE. UNDERSTANDING is the live picture of the room (the Meeting Page from Deep Dive 02) multiplied by the deep knowledge of the company (the Estate Graph from Deep Dive 01). CAPACITY is perfect memory of every meeting, frontier reasoning, real tools that touch real systems, and no fatigue — it never stops paying attention. Multiply the two and you get a participant who knows more and can do more than anyone in the room. The gate is what makes that a gift instead of a menace: it converts raw capacity into *well-timed, proven, wanted* contribution, or into silence.

The capabilities this unlocks

Because Proxy understands the room *and* holds the company's real systems *and* can act on them, it can do a specific, concrete set of things no summarizer or copilot can. Each of these is not a hardcoded feature — it is the same engine pointed at a different moment. They are worth naming because they are what a buyer actually pictures.

- **Catch the wrong number, live.** Someone says "our error rate is 2%." Proxy has already, silently, fetched the live figure the instant "error rate" entered the conversation. It is 3.4%, from production, not the staging dashboard the speaker was reading. Proxy surfaces it with the receipt attached — the source, the timestamp, the exact query — so it is not a contradiction, it is a correction the room can trust on sight.
- **Catch the contradiction the room forgot — even across meetings.** A claim made at minute three is quietly incompatible with one made at minute twenty; or a decision being re-opened today was settled, with reasons, three weeks ago. Proxy holds both and says so. Human memory cannot; Proxy's ledger does.
- **Run the check nobody asked for.** The room is about to commit to shipping Friday. Nobody asks "does anything downstream break?" Proxy checks anyway — walks the blast-radius, finds that invoicing has no fallback for the change, and raises its hand *before* the decision goes final, not in the retro.

- **Prove the number.** Not "I think that's about right" — the actual figure, pulled from the actual system, shown with its source chain. When a claim is challenged, Proxy produces the provenance instantly.
- **Show the blast-radius of a decision.** "If we change checkout, what does it touch?" — answered as a real walk of the estate graph across code *and* org: which services, which teams, which commitments already made elsewhere depend on the thing being changed.
- **Grounded facilitation.** When a group is circling — the same three positions restated with no new information — Proxy can synthesize where they actually agree and disagree, naming the specific point of tension, so the meeting moves. This is facilitation backed by fact, not a generic "let's take a step back."
- **Let the software speak for itself.** The deepest capability, and the one that names the category. A claim about checkout's error rate is a claim *about a participant that can answer for itself*. Proxy is the mouth the company's systems share: "my p99 is 340ms as of 10:02; the 2% figure is from staging, not production." A decision about to hurt a system surfaces as that system defending itself: "this migration breaks three of my consumers." The estate is *in the room*, and Proxy is how it talks.

It serves any kind of meeting

Nothing in this engine knows what *kind* of meeting it is in. There is no "standup mode" and no "board-review mode" — no meeting-type switch anywhere in the code. An engineering design review, a sprint planning session, an incident postmortem, a quarterly business review, a sales call, a hiring debrief, a budget meeting, a board session: Proxy runs the *identical* mechanism in all of them. The only things that differ are which company systems are connected (the estate), what the room is actually saying (the page), and where the user has set the dial. In a design review, untested assumptions become sandbox runs. In sprint planning, an over-optimistic estimate meets "the last three times this took twice as long — here are the receipts." In a sales call, a promise the product can't keep gets whispered privately to the account executive before it's said out loud. Same engine, zero per-type code. That is not a convenience; it is a design property — the engine reasons over the room's live state, and situations differ by their *content*, never by a category the software has to be told.

It works at any size and in any vertical

The engine is size-agnostic and industry-agnostic by construction, because it inherits both properties from the layers beneath it. The Estate Graph is fractal (Deep Dive 01) — a two-person startup or

one product-area of a ten-thousand-person enterprise present the same shape to the engine. And nothing here parses industry: a fintech's payment services, a hospital's data pipelines, and a game studio's engine modules are all just nodes and edges the engine reasons over. A new vertical needs no new code — only a new estate connected.

The user's one control: the five-level dial

Everything above is capability. The user's control over it is a single, legible dial with five levels — from "only speak when it's unarguable and urgent" up to "actively facilitate." The dial is not five different programs; it is five *starting points* on one underlying setting: *how strong does a contribution have to be before the room hears it?* At the low levels Proxy defaults to a **silent hand-raise** — a small glyph on its tile lights up with a one-line reason, and a human opts it in with a tap, so a mistimed interruption is impossible. At the high levels it speaks up on its own. And crucially, the dial *learns*: within whatever level you set, Proxy tunes itself to each specific room from your accept-and-dismiss signals — if this room keeps waving off a kind of contribution, the bar for it quietly rises; if they keep opting one in, it lowers. You set the intent; the room's own reactions do the fine-tuning.

WHY "GENERATE-THEN-FILTER" IS THE WHOLE GAME

An earlier version of this engine gated the *thinking* — it first judged "is this moment worth a thought?" and only then thought. That was the persistent weakness, because judging a hypothetical ("would a thought here be valuable?") is far harder and less reliable than judging a finished thing ("is *this specific drafted contribution* worth saying?"). So we inverted it. Proxy **always forms the thought** — cheaply, on every change — and everything downstream filters concrete, finished artifacts. You filter something real, not a guess about a possibility. This single inversion is why the engine is both more sensitive (it never misses a moment by pre-judging it away) and more precise (it only surfaces things that survived a look at the actual drafted contribution). It is the load-bearing idea of the entire design.

PART B — HOW IT WORKS

The agent design, in plain terms

One mind, two halves, and the four-move loop it runs — before any code.

Proxy's proactive contribution is produced by **one agent with a single identity** — "the most informed participant in the room" — that thinks in **two halves**. This mirrors how people actually think, a distinction psychologists call **System 1 and System 2**: the fast, automatic, always-on half that continuously reacts to what's happening, and the slow, deliberate, effortful half that you summon only when something is worth real thought. Proxy has both, and they are the same identity — not two agents negotiating, one mind at two speeds.

The two halves

The FAST MIND is always on. It is a cheap, persistent session (a Haiku-class model on the Claude Agent SDK, kept warm with prompt caching so it never re-reads its context) that receives *every change to the Meeting Page* — every claim written, every decision moving, every referent resolved. On each change it does one small thing: it composes its best possible contribution *right now*, then decides whether that contribution is worth developing further. Most of the time the honest answer is "nothing" — a five-token output, logged and dropped. That is correct behavior; a good participant is silent most of the time.

The DEEP MIND is the same identity waking up on a frontier model with the full toolbox. It is summoned only when a draft genuinely needs real work to become sayable — a test to run, a live number to check, a precedent to pull, a demo to build. When it wakes it has everything: the **executor sandbox** (a safe workroom where it can run real code), the **unit experts** from Deep Dive 01 (the per-system authorities, which it can call like tools), and web research. It takes the fast mind's rough draft and makes it into a proven artifact — or it discovers the draft was wrong and kills it.

The core loop: DRAFT → DEVELOP → VALIDATE → DELIVER

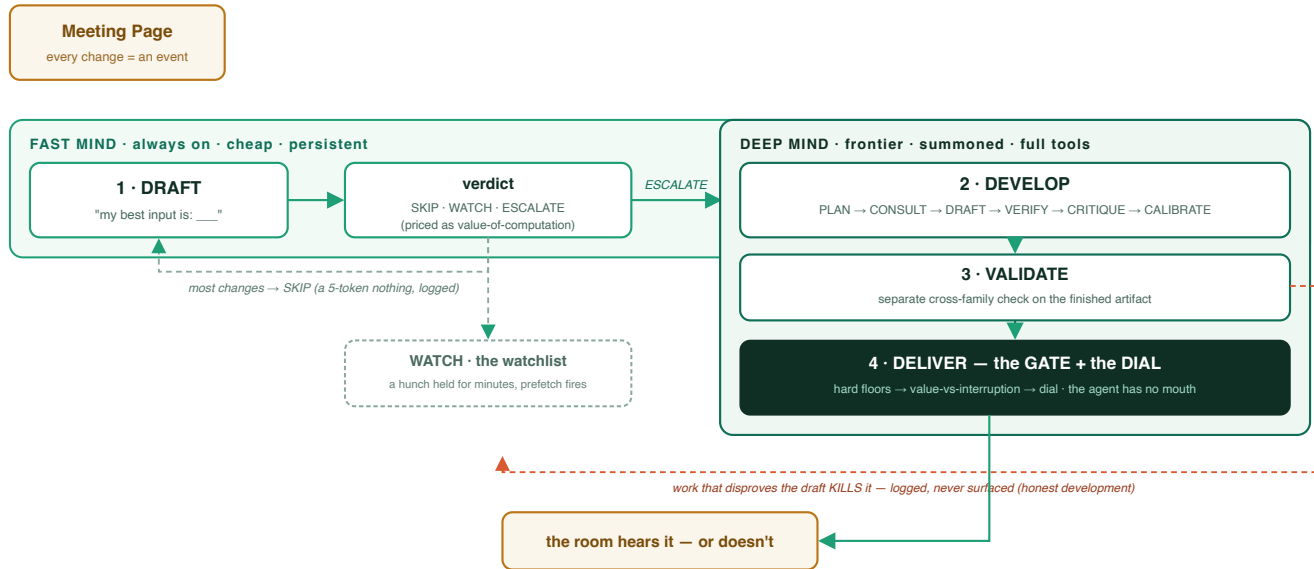


Figure 1 — The two-minds loop. The fast mind drafts on every page change and returns a structured verdict; most are SKIP. WATCH holds a hunch and pre-fetches evidence. ESCALATE wakes the deep mind, which develops, validates, and hands a finished artifact to the deterministic gate — which alone owns the mouth. The orange path is the honesty rule: development that disproves its own draft kills it.

The loop has four moves. Read them once here in plain language; Part C builds each.

DRAFT — form the thought, on every change

On every coalesced change to the Meeting Page, the fast mind composes *the actual candidate contribution* — not "should I say something?" but "if I had to add my single best input right now, it is: ____." What makes "best" computable is a compass: the fast mind aims at **the page's goal and its current blocker** — *what is most in the way of this meeting reaching its goal right now, and what is the strongest thing I could put against it?* The type of contribution — a correction, a missing option, a synthesis — is never chosen from a menu; it is simply the shape of the strongest attack on the blocker. A disputed number makes verification the best input; a missing option makes creativity best; two people talking past each other makes synthesis best; a healthy meeting makes *silence* best. Situation-variability falls out for free because the blocker varies.

The fast mind's output is a tiny structured verdict: a one-line *thought*, a *decision* of SKIP, WATCH, or ESCALATE, a rough *sketch* of what deep work the thought would need, a *priority*, and any *watchlist update*. Most changes return SKIP — logged with a reason, a few tokens, effectively free. The genuinely interesting property is **ESCALATE is not a judgment call, it is a price**. The fast mind escalates only when the expected value of what the deep mind would produce, minus the cost of

running it, beats the value of staying quiet — a formal rule from decision theory called **value-of-computation**. It prices that against a **capability manifest** — an automatically-derived list of what Proxy can actually do in this room (from the tools and experts available at join, calibrated by what has and hasn't paid off in past runs). It never escalates on a hunch it couldn't act on.

WATCH is the third path, and it is what makes catches feel instant. When the fast mind has a hunch that isn't yet actionable — "they'll probably question that number in a minute" — it puts the item on a **watchlist**, a small set of live suspicions held across minutes. Watchlist items expire on their own when the topic dies (topic-liveness), so hunches don't pile up. Crucially, a WATCH *immediately fires a data fetch* in the background, so by the time anyone actually questions the number, the answer is already in hand.

DEVELOP — the deep mind, summoned by the draft's own needs

Escalation hands the deep mind a draft and a sketch of what it needs. The deep mind then runs a disciplined inner think-loop — this is the part that makes Proxy's contributions *right*, not just fast. In order: it **retrieves skill-cards** (approaches that worked in similar past situations, offered as suggestions only); it runs a single **PLAN pass** that generates three things *together* — the reasoning structure it will use, the type of contribution, and a success **rubric** — and *freezes* the rubric; it **consults unit experts** in parallel, only the ones the plan calls for; it **drafts candidate contributions** using verbalized sampling (asking for several candidates with their probabilities, then deliberately mining the unlikely-but-interesting ones); it **verifies** — any number is proven in the sandbox, no exceptions; it runs **exactly one self-critique** pass with a judge from a different model family, scored against that same frozen rubric; and it **calibrates** — its confidence in the drafted answer decides whether it fires at all. The single most important discipline: the rubric is derived once and frozen, so the judge scores against the *same* bar the drafter aimed at. It never re-derives its own success criteria, which is what stops the criteria from quietly drifting to something the answer happens to pass. And development is *honest* — work that disproves the draft **kills it**, logged and never surfaced. Live work is capped at about fifteen seconds of wall-clock; anything slower becomes an async artifact delivered at the next lull, never a late interruption.

VALIDATE — a fresh, skeptical second look

Validation is a *separate, stateless* check on the finished artifact — a different model family from the one that developed it, with its criteria order-swapped and phrased as reasons *not* to surface. It asks four blunt questions: is it **grounded** (receipts attached and checkable), **novel** to the room (not already said or on the page), **material** to the goal, and still **timely**. Judging a finished artifact is far more reliable than judging a hypothetical intention — which is exactly why this check sits here at the end and not up front. When several candidates are strong at once, the tie-break is principled:

stakes × urgency × uniqueness — prefer the contribution only Proxy could have made (the test actually run, the live number actually pulled) over anything an attentive human could have said.

DELIVER — the gate owns the mouth

The critical architectural fact: **the agent has no speaking tools**. It cannot address the room. Only a small, deterministic **gate** can surface anything, and it decides in three ordered stages. First, **hard floors**: never surface an unverified, stale, contradicted, or permission-violating value — full stop. Second, three moments *force* development regardless of any budget: a decision going final, a detected contradiction, and a commitment on a checkable claim. Third, the **dial**: a per-room threshold that weighs the contribution's value against the cost of interrupting, defaulting to a silent hand-raise at low levels. The gate is deterministic code — no model, instant, free, and every one of its decisions is logged with its reason, so a regulated room can lock it and audit it.

Three honest kinds of output

Everything Proxy surfaces is one of three classes, and each has its own rule. **ASSERTIONS** are checkable facts — surfaced only with a passing verification receipt *and* an epistemic stamp saying how it was verified (from a primary source, from a secondary one, or derived by computation); an unverified assertion never reaches the room. **JUDGMENT** is syntheses, options, encouragement, a mock-up — surfaced clearly *labeled* as judgment, and required to reference a concrete entity on the page or in the estate (the specificity filter; generic advice like "have you considered the risks?" dies here). **META** is the instant class for when the moment is moving too fast to check anything: a one-line offer ("I can verify that against billing in ~20 seconds — want it?") or one targeted clarifying question. META carries no factual claim, so it needs no receipt.

PART C — THE BUILD

Every piece, exactly — what it is, how we wire it, how we optimize it

Five runtime surfaces over one ledger. For each: the overview first, then the specifics and how we optimize it. The actual prompts are shown.

The whole engine is **five runtime surfaces reading and writing one shared record** — the candidate ledger. Nothing here is a heavyweight multi-agent framework; coordination is a shared document and events, which is faster and more reliable than agents routing messages to each other. Here is the whole cast before we build each piece.

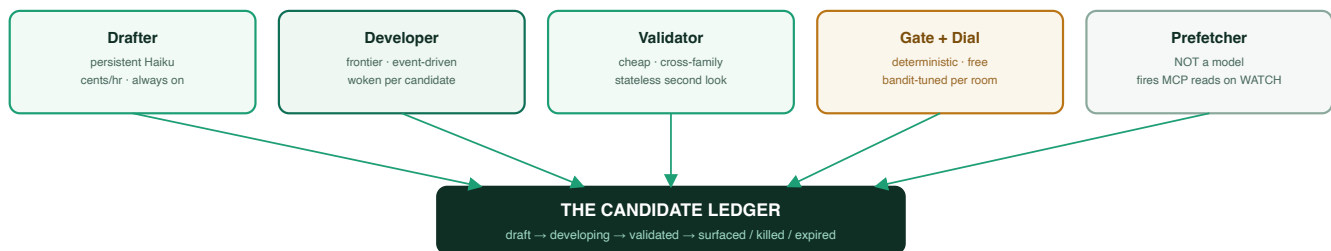


Figure 2 — Five surfaces over one ledger. Each surface reads and writes the same candidate rows; the ledger is simultaneously working memory, the activity feed's data, the audit trail, and the training log. Only the Gate can surface anything.

Surface 1 — The Drafter: the always-on fast mind

Overview. The Drafter is a single persistent session of a cheap Haiku-class model on the Claude Agent SDK. "Persistent" and "prompt-cached" together are what make it affordable: its large, stable context (its instructions, the meeting's goal, the page schema) is cached and never re-read, so each new page change costs only the handful of new tokens in the change itself. It runs all meeting long for cents an hour.

Specifics and optimization. The Drafter receives *coalesced* page diffs — a burst of rapid Scribe edits is merged into one stable snapshot before it drafts, so it thinks on settled states, not on every keystroke (the Coalescer, below). On each snapshot it emits the structured verdict shown here. The escalation decision is priced, not judged — the value-of-computation rule — against the derived capability manifest, so the fast mind never proposes work Proxy can't actually do. The whole thing is

fine-tunable: because every draft-and-outcome pair is logged in the ledger, a small model tuned on the org's own history later replaces the stock one at lower cost *and* higher accuracy.

SYSTEM PROMPT — THE DRAFTER (FAST-MIND MICRO-THOUGHT, ILLUSTRATIVE)

You are the most informed participant in this room, forming what you WOULD say. On each Meeting Page change, compose your single best possible contribution — never "should I speak?", always "if I had to add my best input now, it is: ____".

Aim at the page's GOAL and its CURRENT BLOCKER:

what most blocks this meeting reaching its goal right now, and what is the strongest thing I could put against it? The contribution TYPE is the shape of that answer (a correction, a missing option, a synthesis) — never a menu.

You may act only within the capability manifest you were given. ESCALATE only when the expected value of the deep mind's result, minus its cost, beats staying quiet (value-of-computation). Most changes are SKIP — that is correct.

Return ONLY this object:

```
{ thought:      "<=1 line, your best input or why none",
  decision:      SKIP | WATCH | ESCALATE,
  sketch:        "the deep work this would need (a test? a live number? a
                  precedent?) — the property that summons the deep mind",
  priority:       0.0-1.0,
  watchlist_update: [ {suspicion, expires_when: topic-liveness} ] }
```

SKIP is a 5-token answer. WATCH holds a hunch and fires a prefetch. ESCALATE carries the sketch. Never invent an entity; reference only what is on the page.

The capability manifest — why escalation is priced against something real

Overview. Value-of-computation needs to know what the deep mind is *able* to return before it's worth waking it. That "what can I do here" list is the capability manifest — and it is *derived*, never authored. At join, it is assembled from the tool and expert registry available in this room (which systems are connected, which experts are on the bench, which action classes exist). Then it is continuously calibrated by run history: if a kind of deep work has repeatedly failed to land in this room, its expected value is marked down; if it reliably pays off, up. So the fast mind's escalation math is grounded in what Proxy can *actually accomplish here*, not a generic optimism.

The watchlist — hunches held across time

Overview. Not every useful thought is actionable the instant it forms. The watchlist is where the fast mind parks a suspicion — "this staging number will probably get challenged" — and holds it for minutes. Two optimizations make it cheap and clean: each item carries an **expiry tied to topic-liveness**, so when the conversation moves off the topic the hunch drops itself and never

accumulates; and each item *immediately triggers the Prefetcher*, so the evidence is fetched into a ready bundle before it is ever needed. This is the mechanism behind catches that feel instant.

Surface 2 — The Developer: the deep mind's inner cognition

Overview. The Developer is the same identity waking on a frontier model, event-driven — it exists only when a candidate is being developed, so it costs nothing between wakes. It runs the disciplined seven-step think-loop that turns a rough draft into a proven artifact (or kills it). The design principle underneath the whole loop: **strategy, criteria, and contribution type are one generative act, not three lookups**, and skill memory is a prior that act conditions on.

Specifics. The loop, in order:

- **RETRIEVE.** Pull the top few skill-cards — distilled approaches that worked in similar past situations — matched by an embedding of the *situation*, not a category label, and gated by recency and past acceptance. They are *suggestions*, never auto-applied branches.
- **PLAN — the single derivation pass.** One frontier call reads the fast-mind sketch, the meeting-state snapshot, the manifest, and the retrieved skills, and generates *together*: (a) the **reasoning structure** — composed from a fixed menu of thinking primitives (verify-this-claim, enumerate-options, synthesize-positions, build-artifact, decompose-and-plan) the way a person assembles an approach; (b) the **contribution type**, diagnosed from group-process principles the model holds as knowledge; (c) the **success rubric**, derived from the specific context; and it sets an **effort** level and a **horizon** flag of live or async. The rubric is **frozen** here. This fusion matters: a wrong-number situation yields {verify-structure, correction-type, correctness-first rubric} as one coherent object, because the three are mutually constraining.
- **CONSULT.** Call only the unit experts the plan names, in parallel, as tools with a shared scratchpad — the per-system authorities from Deep Dive 01, handed only the relevant slice of meeting state to keep tokens low.
- **DRAFT.** Generate several candidate contributions with their probabilities in one call (verbalized sampling), deliberately mining the lower-probability tail for the non-obvious answer — all scored against the *frozen* rubric.
- **VERIFY.** A hard floor: any number or checkable claim is proven in the sandbox, unconditionally. No exceptions, ever.
- **SELF-CRITIQUE.** Exactly one pass — a judge from a *different model family*, scoring the candidates against the *same* frozen rubric. It never re-derives the rubric; that single discipline is what kills criteria-drift. (One pass, not a loop — the evidence is clear that repair has sharply diminishing returns.)

- **CALIBRATE.** Confidence, conditioned on the actual drafted answer, decides whether the candidate fires at all — low confidence or contradicting sources means silence.

Then it exits to the gate. Throughout, the Developer re-drafts as it learns (the draft is a starting direction, not a commitment), respects the ~15-second live wall-clock cap (overflow becomes an async artifact), and writes two or three lines back into the shared meeting memory — its read of where the meeting is going — enriching every later draft without constraining one.

SYSTEM PROMPT — THE DEVELOPER'S PLAN PASS (THE SINGLE DERIVATION, ILLUSTRATIVE)

A draft has escalated to you. Before you do any work, run ONE derivation pass that composes your whole approach at once. Read: {fast-mind sketch, meeting-state snapshot, capability manifest, retrieved skill-cards}. Then GENERATE, together:

```

structure:  compose an approach from these primitives (pick + order + adapt):
            verify-this-claim · enumerate-options · synthesize-positions ·
            build-artifact · decompose-and-plan · do-nothing
            (compose per task – do NOT run a fixed pipeline)

type:       the contribution's shape, diagnosed from group-process principles:
            circling → synthesis; missing viewpoint → reframe; wrong number
            about to anchor a decision → correction. Derive it; never look it up.

rubric:     the success criteria THIS contribution must meet. Derive from the
            context. It MUST include the floors – grounded, specific, permitted,
            non-length, and explicit negative criteria ("reasons NOT to surface").
            You may ADD dimensions; you may not remove a floor.
            >>> This rubric is FROZEN now. The drafter and the judge both score
            against THIS exact rubric. Neither re-derives it. <<<

effort:     how much compute this deserves (bounded – one contribution).
horizon:    live | async  (if this is long-horizon work, mark async – it
                        becomes a deliverable, never a live interjection)

```

Retrieved skills are SUGGESTIONS – adopt, adapt, or ignore in context.
Then: CONSULT (only named experts) → DRAFT (verbalized sampling vs the frozen rubric) → VERIFY (numbers sandbox-proven, floor) → one CRITIQUE (cross-family, same frozen rubric) → CALIBRATE. Work that DISPROVES the draft KILLS it – log it.

Skill memory — how accepted approaches compound

Overview. When a developed contribution is *accepted* in the room, a background pass distills the *approach* — the composed structure, the derived rubric, the retrieval pattern — into a **skill card** (a Claude Agent Skill: a small folder the deep mind can retrieve and inject). Cards are keyed by an embedding of the situation, not a label, so they are retrieved by *similarity*, and they feed PLAN as priors it may adopt, adapt, or ignore.

The optimization that keeps it honest. Agent Skills have no built-in staleness handling, so we build one: each card carries a last-validated timestamp and an accept/dismiss counter, and a card that gets repeatedly dismissed in shadow is **demoted**. Retrieval is gated on similarity *and* recency *and* acceptance, so stale advice quietly falls out of circulation instead of poisoning future plans.

Surface 3 — The Validator: the fresh skeptical check

Overview. The Validator is a cheap, stateless call from a different model family than the Developer. It never sees how the artifact was made — it just judges the finished thing on grounded / novel / material / timely, with the criteria order-swapped and phrased as reasons *not* to surface. Being stateless and cross-family is the point: it has none of the developer's investment in its own answer, so it catches the developer's blind spots.

Optimization. Order-swapping the criteria and forcing negative framing are the standard, measured fixes for the biases that make LLM judges unreliable (position bias, self-flattery, sycophancy). The tie-break when several candidates pass — stakes × urgency × uniqueness — is deterministic arithmetic, not another model call, so it is free and auditable.

Surface 4 — The Gate and the Dial: the only mouth

Overview. The Gate is deterministic code — no model, instant, free. It is the *only* thing in the whole engine that can surface anything; the agent has no speaking tools by design. It decides in three ordered stages: hard floors, then the forced-development moments, then the dial.

THE THREE UNCONDITIONAL HARD FLOORS — FORCED DEVELOPMENT REGARDLESS OF SPEND

Three moments override every budget, every threshold, every judgment call, in code: **(1) a decision going final** — every affected system is heard before the room commits (blast-radius as a stakeholder check); **(2) a detected contradiction** — the system's right to correct the record about itself; **(3) a commitment on a checkable claim** — "you just promised 99.9%; I run at 99.7% with no EU redundancy." These are the software's rights, made mechanical. When one fires, development happens no matter what the cost envelope says. Everything else is discretionary; these are not.

The dial, specifically. The five levels are not five programs — they are five *starting priors* on one intervention threshold. From that starting point, a **per-room contextual bandit** tunes the actual threshold from accept and dismiss signals, so each room converges to its own tolerance. The parameters are interpretable and every decision is logged with its explanation, so a regulated room can lock the dial and audit it. At low levels the default surfacing mode is a **silent bid-to-speak** —

the tile glyph lights, a one-line reason appears, a human taps to opt it in — which is the evidence-backed cure for the mistimed-interruption failure. Only at high levels does it surface on its own. One surfaced act per window; a human's direct ask always outranks anything proactive.

Surface 5 — The Prefetcher: why catches feel instant

Overview. The Prefetcher is *not a model* — it is the cheapest and most important optimization in the engine. When a draft marks something WATCH, the Prefetcher immediately fires deterministic reads through the connected systems (the live value, its definition, its recent history) into a small evidence bundle stamped with a short time-to-live. It is data-fetching, not thinking, so it is nearly free. The payoff: by the time someone finishes *saying* the wrong number, the right one is usually already fetched and sitting in the bundle, so the deep mind's "check" is a cache hit and the catch lands in about a second instead of ten.

The Candidate Ledger — the glue

Overview. One table binds all five surfaces. Every draft is a row that advances through `draft → developing → validated → surfaced / killed / expired`, and only the surface that owns a stage may advance it (so parallel drafts and parallel developments never clobber each other). The ledger is four things at once: the engine's working memory, the data behind the activity feed people see (Deep Dive 06), the audit trail, and the training log that both the fine-tuned drafter and the skill-distiller learn from. Because every silence is a logged SKIP with a reason, *every non-contribution is queryable afterward* — you can always ask "why didn't you say anything at minute 14?" and get a real answer.

The estate as participant — the drafter's second stance

Overview. The proactive engine is how the company's software *attends the meeting*. This is a stance layered onto the exact same machinery, not new code. Alongside "what would the best-informed participant add?", the Drafter asks a second question: *what would the systems in this room say right now, if they could speak?* A claim about checkout's error rate is a claim about a participant that can answer for itself. Drafts framed as a system's own perspective are concrete by construction — the specificity filter becomes nearly automatic — and they surface signed in the first person ("*billing-service: my p99 is 340ms as of 10:02*"). The three hard floors, read in this light, are the software's **rights**: the right to correct the record about itself, the right to be heard before a decision commits it, and the right to confirm it can honor a promise being made in its name. The systems are joining the meeting in human form, and Proxy is the one human-shaped mouth they share.

The seven agentic abilities — all riding pieces already built

These are not new subsystems; each is a consequence of the ledger, the two minds, the experts, and the sandbox already described. Named briefly because they are what makes the engine feel alive.

- **Living drafts** — candidates persist in the ledger; a hunch at minute 12 is strengthened, updated, or expired by later diffs and matures at minute 30 without being recomputed from scratch.
- **Self-interruption** — the Developer checks the deadline mid-work; if the room moved on, it stops, logs, and banks the partial result rather than arriving late.
- **Whisper-first** — sensitive catches (a misquoted number, a self-contradiction) route privately to the relevant person's panel first; they decide if the room hears it. Default for corrections at low dial levels.
- **Parallel development** — two or three candidates develop at once (envelope-governed); the gate still serializes surfacing. Capacity no human in the room has.
- **Second life for expired candidates** — validated-but-late work lands in the close-of-meeting reveal and per-person follow-ups; the moment can pass, the value doesn't.
- **Quiet human asks** — when development hits a gap only a human can close, one private question goes to the mapped owner (the unknowns loop from Deep Dive 01, live).
- **Series memory** — a recurring meeting pre-seeds its ledger and memory from the previous instance, so Proxy walks into standup #12 already carrying every open thread from #11.

And the property that ties Part A's promise to Part C's build: **the identical configuration serves every meeting type**. Engineering review, sales call, board session — nothing branches on type. Only the dial and the connected estate differ; the mechanism is byte-for-byte the same.

How the whole engine stays cheap, fast, and trustworthy

- **The Coalescer.** The Drafter thinks on *stable page snapshots*, not on every micro-edit — bursts of Scribe writes coalesce into one diff before drafting, cutting the fast mind's call volume sharply. It is deterministic edit-magnitude logic, not another model. The one exception: the hard floors bypass the coalescer, so a decision going final is never delayed by a coalescing window.
- **No fixed proactive sub-budget.** There is deliberately no "the proactive engine gets \$X/hour" cap. There is only a *soft* overall meeting envelope the agent can see and weigh, plus the wall-clock cap — which is latency physics (live voice versus a deliverable), never a thinking budget. We never make Proxy dumber to hit a number.
- **Cost comes down by cutting plumbing, never brains.** Roughly 60% of a meeting's cost is transport and speech-to-text — pipes that scale down with zero quality impact. *All* the intelligence combined is cents on the dollar. So every cost lever we pull is on the plumbing; the judgment core stays on frontier models, non-negotiable.
- **Ship dark.** The whole engine launches in shadow: it drafts, develops, validates, and logs exactly what it *would* have said, while the reactive path delivers the visible value. It is promoted to live surfacing **room by room**, only as its shadow precision proves out — which also closes the one honest evidence gap from the research (per-room tuning is proven for notifications, unproven for live barge-in) before anything speaks.
- **Per-room learning.** The gate threshold is initialized with a distribution-free method (conformal) and then tuned per room by the contextual bandit — so each room's tolerance is learned from its own reactions, not assumed.
- **The flywheel makes it cheaper every month.** Because every draft-and-outcome pair is logged, a small drafter fine-tuned on the org's own data replaces the stock model at several-fold lower cost *and* higher accuracy. Quality rises while cost falls — the compounding advantage of logging everything from day one.
- **The number.** This engine runs at roughly **\$0.20–0.35 per hour** (about \$0.50 in a worst-case storm of activity), inside a full-meeting envelope of ~\$1.05–1.75/hr at v0 quality, engineering down toward ~\$0.45–0.80/hr at scale *with better* quality than v0.

Done when

This engine is complete when: the drafter produces a candidate or an explicit NOTHING on every coalesced page change with no semantic precondition, at cheap-model cost; a draft that declares a development need demonstrably summons the right capability with no capability list hardcoded anywhere; the three hard floors force development unconditionally on replay regardless of spend; no fixed proactive sub-budget exists, only the soft envelope and the live wall-clock cap; assertions never surface without a passing receipt, judgment never without its label and a concrete entity, and META offers carry zero factual claims; the prefetcher measurably turns a WATCH into sub-second evidence; simple catches bid inside a few seconds and developed catches inside about twenty on real meetings; expired candidates provably die unspoken and resurface only in post-meeting delivery; the dial changes surfacing at every level while the drafting logs stay identical across levels; whisper-first routes corrections privately by default at low levels; every silence is queryable afterward with its reason; the shadow record supports a per-room precision read before any live promotion; the drafter fine-tune loop runs on ledger data with measurable per-room improvement; a reactive ask flows through the same ledger and machinery as a proactive candidate with no duplicated engine code; and the identical configuration serves engineering, sales, finance, hiring, and board meetings with only the dial and the connected estate differing.